Moveit Scene Design

Moveit Scene Design
- 1. Usage Environment
- 2. Driving the Real Machine
  - 2.1 Starting the Real Device
- 3. Starting MoveIt2
  - 3.1 Scenario Design
  - 3.2. Startup Command
  - 3.3. Source Code Analysis

1. Usage Environment

ROS2: Humble

Enter the commands in the tutorial into the accompanying virtual machine to run the function.

2. Driving the Real Machine

Driving the real machine is achieved by subscribing to the Moveit2 `/joint_states` topic, converting the joint state information of the robotic arm into control data for the real robotic arm.

Note: Since the real robotic arm does not have obstacle avoidance capabilities, some poses may encounter obstacles; therefore, the planned robotic arm movements should be as reasonable as possible and avoid locations with obstacles.

(It is recommended to use preset poses for demonstrating the driving of the real machine.)

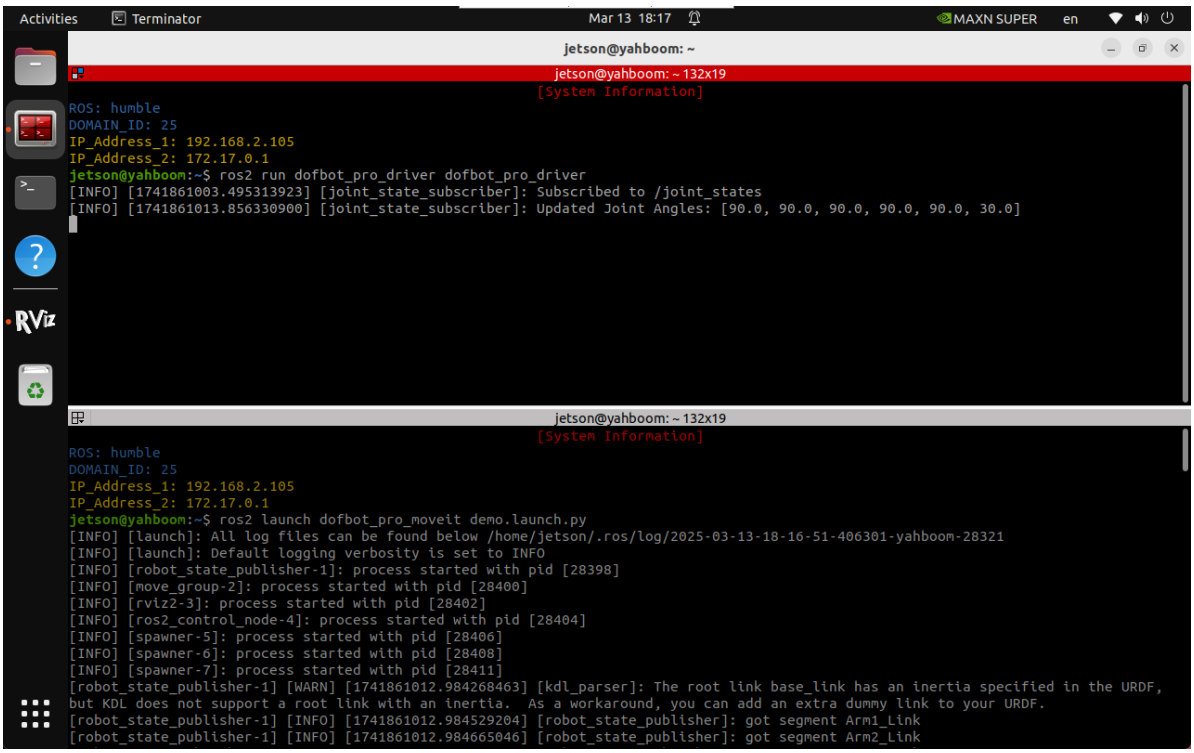
The motherboard and virtual machine need to be configured for multi-machine communication. The robotic arm and virtual machine need to be on the same network, and the virtual machine's `ros_domain_id` must be consistent with the `ros_domain_id` in Docker. (Both devices are pre-configured with the same ros_domain_id at the factory; only the virtual machine and the robotic arm need to be on the same network.)

2.1 Starting the Real Device

```
# Start this command in Docker:
ros2 run dofbot_ctrl SimulateToArm
```

3. Starting MoveIt2

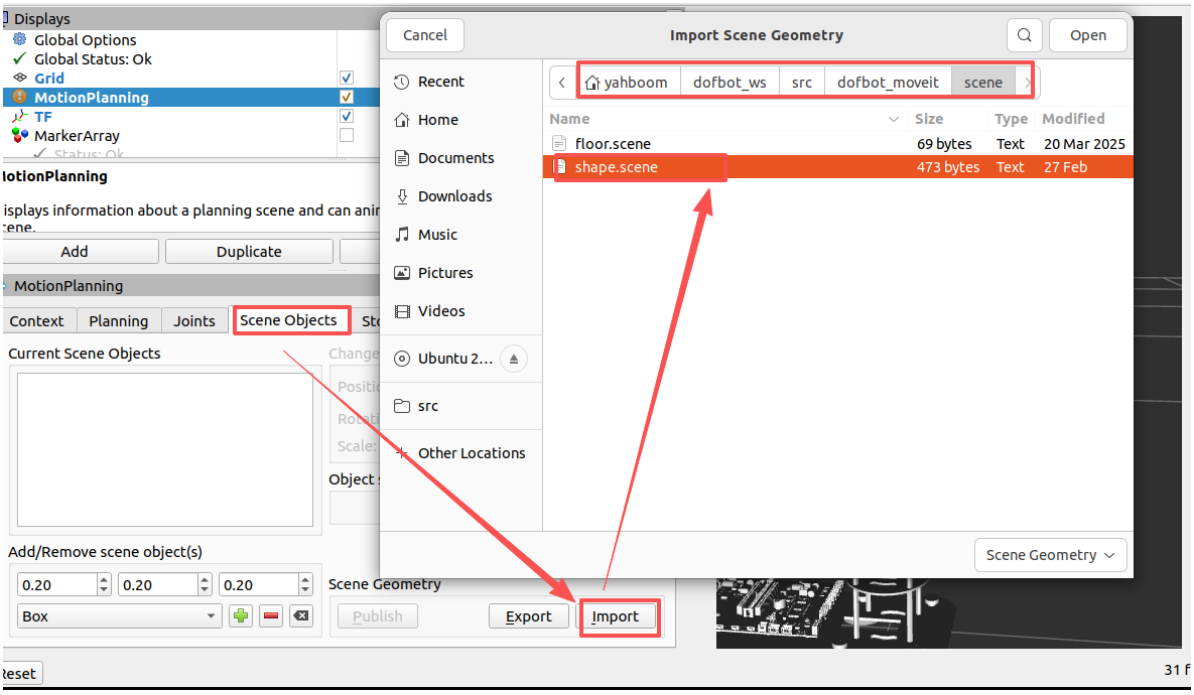
```
# Start this command in the virtual machine:
ros2 launch dofbot_moveit demo.launch.py
```



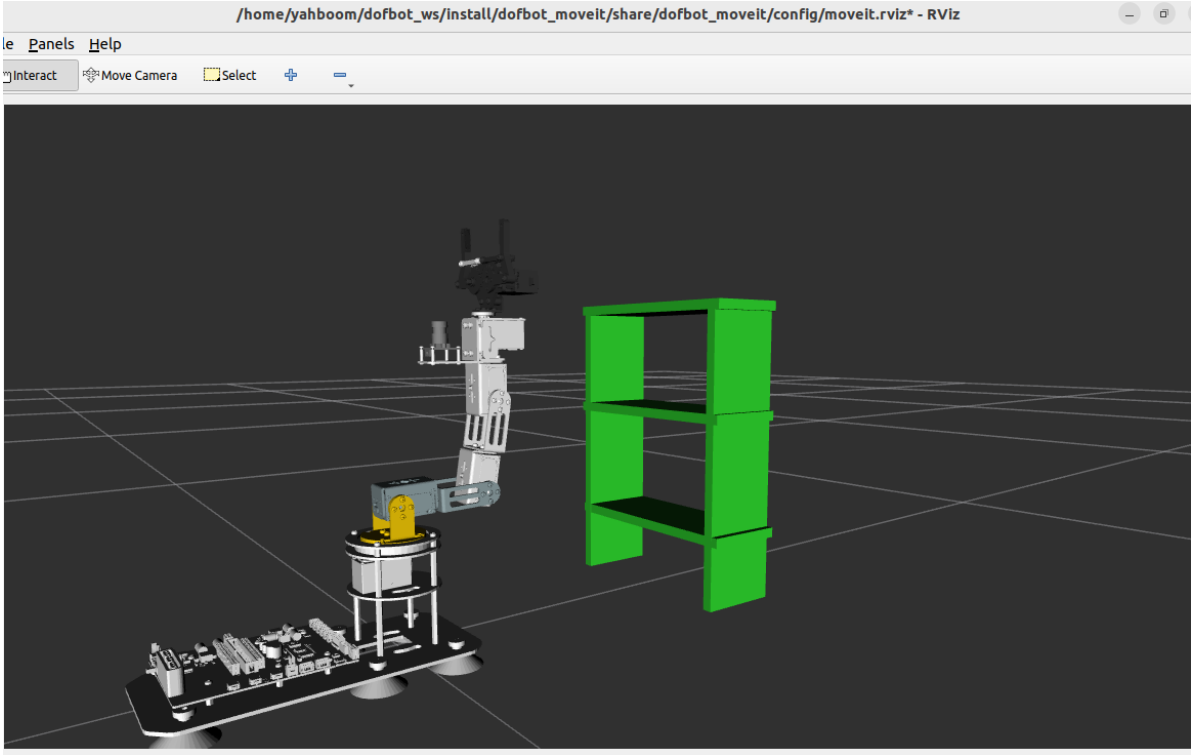
3.1 Scenario Design

In RViz, there is a three-tiered shelf. After the robotic arm picks up a cylindrical object, it plans to place it on the upper and lower tiers in the middle, simulating the object placement action.

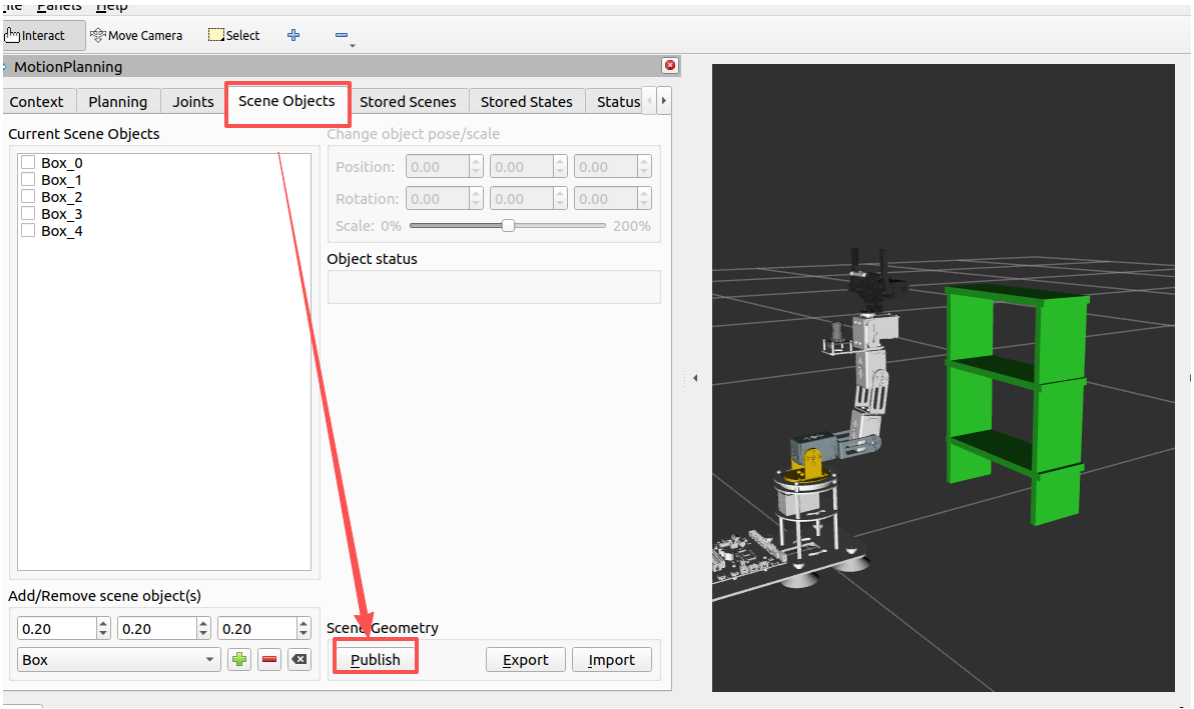
As shown in the figure below, add a three-tier shelf:



Click "Open" to complete the addition of the shelf, as shown in the figure below:



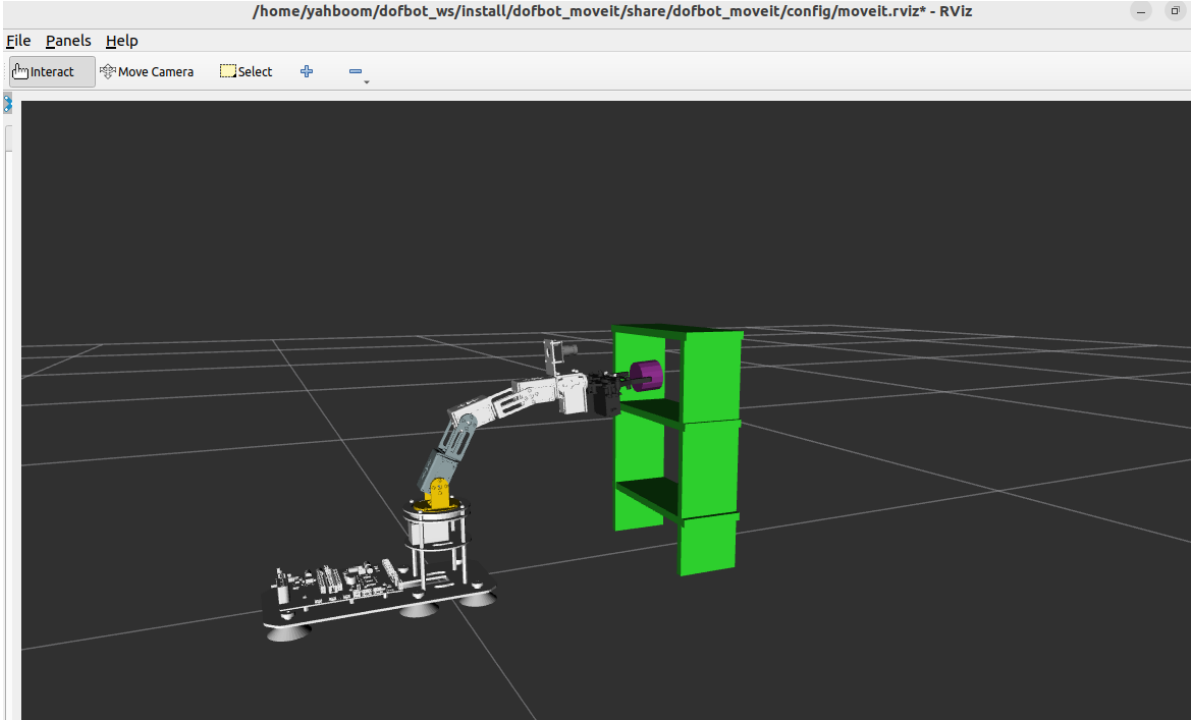
Then, click "Publish" to publish the relevant information regarding the shelf:



3.2. Startup Command

For the robotic arm to successfully load in MoveIt—and specifically once the message `You can start planning now!` appears—execute the following command:

```
# Execute this command within the virtual machine:
ros2 run dofbot_moveit set_scene
```



In RViz, a cylindrical object will be added to the robotic arm; the arm will then execute a movement sequence to simulate the scenario of placing objects on the upper and lower tiers of the shelf.

3.3. Source Code Analysis

Source code path:

```
# Within the virtual machine:
~/dofbot_ws/src/dofbot_moveit/src/set_scene.cpp
```

```
#include <rclcpp/rclcpp.hpp>
#include <moveit/move_group_interface/move_group_interface.h>
#include <moveit/planning_scene_interface/planning_scene_interface.h>
#include <geometry_msgs/msg/pose.hpp>
#include <random>

class RandomMove : public rclcpp::Node
{
public:
  RandomMove()
    : Node("random_target_move")
  {
    // Initialize other content
    RCLCPP_INFO(this->get_logger(), "Initializing RandomMoveIt2Control.");
  }

  void initialize()
  {
    int max_attempts = 5;  // Maximum number of planning attempts
    int attempt_count = 0;  // Current attempt count
    // Initialize move_group_interface_ and planning_scene_interface_ in this
function_
    move_group_interface_ =
std::make_shared<moveit::planning_interface::MoveGroupInterface>
(shared_from_this(), "arm_group");
```

```
    planning_scene_interface_ =
std::make_shared<moveit::planning_interface::PlanningSceneInterface>();

    move_group_interface_->setNumPlanningAttempts(10);  // Set the maximum
number of planning attempts to 10
    move_group_interface_->setPlanningTime(5.0);         // Set the maximum
planning time per attempt to 5 seconds.

    moveit_msgs::msg::CollisionObject object_to_attach;
    object_to_attach.id = "cylinder1";
    //Add Cylinder
    shape_msgs::msg::SolidPrimitive cylinder_primitive;
    cylinder_primitive.type = cylinder_primitive.CYLINDER;
    cylinder_primitive.dimensions.resize(2);
    cylinder_primitive.dimensions[cylinder_primitive.CYLINDER_HEIGHT] = 0.03;
    cylinder_primitive.dimensions[cylinder_primitive.CYLINDER_RADIUS] = 0.02;
    //Get End Position
    object_to_attach.header.frame_id = move_group_interface_-
>getEndEffectorLink();
    geometry_msgs::msg::Pose grab_pose;
    grab_pose.orientation.w = 1.0;
    grab_pose.position.z = 0.09;
    grab_pose.position.x = 0.0;
    //Add a cylinder to the end position.
    object_to_attach.primitives.push_back(cylinder_primitive);
    object_to_attach.primitive_poses.push_back(grab_pose);
    object_to_attach.operation = object_to_attach.ADD;
    planning_scene_interface_->applyCollisionObject(object_to_attach);

    RCLCPP_INFO(this->get_logger(), "Attach the object to the robot");
    std::vector<std::string> touch_links;
    touch_links.push_back("Llink2");
    touch_links.push_back("Rlink2");
    move_group_interface_->attachObject(object_to_attach.id,
"Gripping_point_Link", touch_links);


    // Plan Path
    moveit::planning_interface::MoveGroupInterface::Plan my_plan;

    move_group_interface_->setNamedTarget("init");
    bool success = (move_group_interface_->plan(my_plan) ==
moveit::core::MoveItErrorCode::SUCCESS);
    if (success)
    {
        RCLCPP_INFO(this->get_logger(), "Arm down successed.");
        move_group_interface_->execute(my_plan);
    }
    else
    {
        RCLCPP_ERROR(this->get_logger(), "Arm down failed!");
    }
    //Move the robotic arm to...
    move_group_interface_->setNamedTarget("down");
    success= (move_group_interface_->plan(my_plan) ==
moveit::core::MoveItErrorCode::SUCCESS);
    if (success)
    {
```

```
        RCLCPP_INFO(this->get_logger(), "Init arm successed.");
        move_group_interface_->execute(my_plan);
    }
    else
    {
        RCLCPP_ERROR(this->get_logger(), "Init arm failed!");
    }

    while (attempt_count < max_attempts)
    {
        attempt_count++;
        std::vector<double> target_joints = {0.0, 0.52, 0.786, 0.20, 0}; //
Target joint angles (units: radians)
        // Set target joint space values
        move_group_interface_->setJointValueTarget(target_joints);
            // Plan Path
        moveit::planning_interface::MoveGroupInterface::Plan my_plan;
        bool success = (move_group_interface_->plan(my_plan) ==
moveit::core::MoveItErrorCode::SUCCESS);
        if (success)
        {
        RCLCPP_INFO(this->get_logger(), "Planning succeeded, moving the arm.");
        move_group_interface_->execute(my_plan);
        break;
        }
        else
        {
        RCLCPP_INFO(this->get_logger(), "Planning failed!");
        }
    }


    while (attempt_count < max_attempts)
    {
        attempt_count++;
        // Set target joint space values
        move_group_interface_->setNamedTarget("init");
            // Plan Path
        moveit::planning_interface::MoveGroupInterface::Plan my_plan;
        bool success = (move_group_interface_->plan(my_plan) ==
moveit::core::MoveItErrorCode::SUCCESS);
        if (success)
        {
          // Visualize the trajectory in RViz
        RCLCPP_INFO(this->get_logger(), "Planning succeeded, moving the arm.");
        move_group_interface_->execute(my_plan);
        attempt_count = 0;
        break;
        }
        else
        {
        RCLCPP_INFO(this->get_logger(), "Planning failed!");
        }
    }

    RCLCPP_INFO(this->get_logger(), "Exit!");

  }
```

```
private:
  std::shared_ptr<moveit::planning_interface::MoveGroupInterface>
move_group_interface_;
  std::shared_ptr<moveit::planning_interface::PlanningSceneInterface>
planning_scene_interface_;
};

int main(int argc, char **argv)
{
  rclcpp::init(argc, argv);
  auto node = std::make_shared<RandomMove>();

  // Lazy Initialization
  node->initialize();

  rclcpp::spin(node);
  rclcpp::shutdown();
  return 0;
}
```